Import required libraries

In [10]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

In [11]:
```
data = pd.read_csv(r'C:\Users\user\FSDS_Adv_Gene_AI_&Agentic_AI\Machine Learning\Regression\M
data
```

Out[11]:

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino
...	...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	1	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	1	chevy s-10

398 rows × 10 columns

In [12]:
```
data.head()
```

Out[12]:

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino

In [13]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 10 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   mpg       398 non-null    float64
 1   cyl       398 non-null    int64
 2   disp      398 non-null    float64
 3   hp        398 non-null    object
 4   wt        398 non-null    int64
 5   acc       398 non-null    float64
 6   yr        398 non-null    int64
 7   origin    398 non-null    int64
 8   car_type  398 non-null    int64
 9   car_name  398 non-null    object
dtypes: float64(3), int64(5), object(2)
memory usage: 31.2+ KB
```

In [14]:
```
data.isna().sum()
```

Out[14]:
```
mpg         0
cyl         0
disp        0
hp          0
wt          0
acc         0
yr          0
origin      0
car_type    0
car_name    0
dtype: int64
```

In [15]:
```
data = data.drop(['car_name'], axis = 1)
data
```

Out[15]:

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type
0	18.0	8	307.0	130	3504	12.0	70	1	0
1	15.0	8	350.0	165	3693	11.5	70	1	0
2	18.0	8	318.0	150	3436	11.0	70	1	0
3	16.0	8	304.0	150	3433	12.0	70	1	0
4	17.0	8	302.0	140	3449	10.5	70	1	0
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	1
394	44.0	4	97.0	52	2130	24.6	82	2	1
395	32.0	4	135.0	84	2295	11.6	82	1	1
396	28.0	4	120.0	79	2625	18.6	82	1	1
397	31.0	4	119.0	82	2720	19.4	82	1	1

398 rows × 9 columns

In [16]:
```
data = data.replace('?', np.nan)
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   mpg       398 non-null    float64
 1   cyl       398 non-null    int64  
 2   disp      398 non-null    float64
 3   hp        392 non-null    object 
 4   wt        398 non-null    int64  
 5   acc       398 non-null    float64
 6   yr        398 non-null    int64  
 7   origin    398 non-null    int64  
 8   car_type  398 non-null    int64  
dtypes: float64(3), int64(5), object(1)
memory usage: 28.1+ KB
```

In [20]:
```
data.isna().sum()
```

Out[20]:
```
mpg               0
cyl               0
disp              0
hp                6
wt                0
acc               0
yr                0
car_type          0
origin_america    0
origin_asia       0
origin_europe     0
dtype: int64
```

This line converts the values in the hp (horsepower) column into numbers.

data['hp'] = pd.to_numeric(data['hp'], errors = 'coerce')

data['hp'] → Selects the hp column from the DataFrame.

pd.to_numeric() → Converts values to numeric (integers or decimals).

errors='coerce' → If a value cannot be converted into a number, it is replaced with NaN (Not a Number), which represents a missing value.

In [27]:
```
data['hp'] = pd.to_numeric(data['hp'], errors = 'coerce')
data.isna().sum()
```

Out[27]:
```
mpg              0
cyl              0
disp             0
hp               6
wt               0
acc              0
yr               0
car_type         0
origin_america   0
origin_asia      0
origin_europe    0
dtype: int64
```

In [19]:
```
data['origin'] = data['origin'].replace({1: 'america', 2: 'europe', 3: 'asia'})
data = pd.get_dummies(data,columns = ['origin'])
```

In [22]:
```
data.head()
```

Out[22]:

	mpg	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_europe
0	18.0	8	307.0	130	3504	12.0	70	0	True	False	False
1	15.0	8	350.0	165	3693	11.5	70	0	True	False	False
2	18.0	8	318.0	150	3436	11.0	70	0	True	False	False
3	16.0	8	304.0	150	3433	12.0	70	0	True	False	False
4	17.0	8	302.0	140	3449	10.5	70	0	True	False	False

In [28]:
```
data = data.apply(lambda x: x.fillna(x.median()), axis = 0)
data.isnull().sum()
```

Out[28]:
```
mpg              0
cyl              0
disp             0
hp               0
wt               0
acc              0
yr               0
car_type         0
origin_america   0
origin_asia      0
origin_europe    0
dtype: int64
```

2 Creading model

Import Linear Regression Machine Learning Libraries

In [32]:
```
from sklearn import preprocessing
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
```

In [33]:
```
X = data.drop(['mpg'], axis = 1) # Independent Variable
Y = data[['mpg']]                # Dependent variable
```

Scaling the datset

In [34]:
```
X_s = preprocessing.scale(X)
X_s = pd.DataFrame(X_s, columns = X.columns)  #converting scaled data into dataframe

Y_s = preprocessing.scale(Y)
Y_s = pd.DataFrame(Y_s, columns = Y.columns)  #ideally train, test data should be in columns
```

Split train and test the dataset

In [44]:
```
X_train, X_test, Y_train, Y_test = train_test_split(X_s,Y_s, test_size = 0.30, random_state =
X_train.shape
```

Out[44]:
```
(278, 10)
```

In [45]:
```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

Fit simple linear regression model and find coeficient

In [46]:
```
regression_model = LinearRegression()
regression_model.fit(X_train, Y_train)

for idx, col_name in enumerate(X_train.columns):
    print('The coefficient for {} is {}'. format(col_name, regression_model.coef_[0][idx]))

intercept = regression_model.intercept_[0]
print('The intercept is {}'.format(intercept))
```

```
The coefficient for cyl is 0.3210223856916096
The coefficient for disp is 0.32483430918483935
The coefficient for hp is -0.22916950059437635
The coefficient for wt is -0.7112101905072296
The coefficient for acc is 0.014713682764191155
The coefficient for yr is 0.3755811949510742
The coefficient for car_type is 0.38147694842330954
The coefficient for origin_america is -0.07472247547584211
The coefficient for origin_asia is 0.04451525203567797
The coefficient for origin_europe is 0.04834854953945393
The intercept is 0.019284116103639705
```

Regularization Ridge Regression

In [47]:
```
ridge_model = Ridge(alpha = 0.3)
ridge_model.fit(X_train, Y_train)

print('Ridge model coef: {}'.format(ridge_model.coef_)) # dataset has 10 col so, we get 10 co
```

```
Ridge model coef: [ 0.31649043  0.31320707 -0.22876025 -0.70109447  0.01295851  0.37447352
  0.37725608 -0.07423624  0.04441039  0.04784031]
```

Regularization Lasso Regression

In [48]:
```
lasso_model = Lasso(alpha = 0.1)
lasso_model.fit(X_train, Y_train)

print('Lasso Model coef: {}'.format(lasso_model.coef_)) # dataset has 10 col so, we get 10 co
```

```
Lasso Model coef: [-0.         -0.         -0.01690287 -0.51890013  0.          0.28138241
  0.1278489  -0.01642647  0.          0.        ]
```

Score Comparision

In [51]:
```
#Model score - r^2 or coeff of determinant
#r^2 = 1-(RSS/TSS) = Regression error/TSS

# Simple Linear model

print(regression_model.score(X_train, Y_train))
print(regression_model.score(X_test, Y_test))


print('************************')
#Ridge
print(ridge_model.score(X_train, Y_train))
print(ridge_model.score(X_test, Y_test))


print('************************')
Lasso
print(lasso_model.score(X_train, Y_train))
print(lasso_model.score(X_test, Y_test))
```

```
0.8343770256960538
0.8513421387780067
************************
0.8343617931312616
0.8518882171608504
************************
0.7938010766228453
0.8375229615977083
```